

线性神经网络

第 4 讲：无隐藏层网络

范一苇

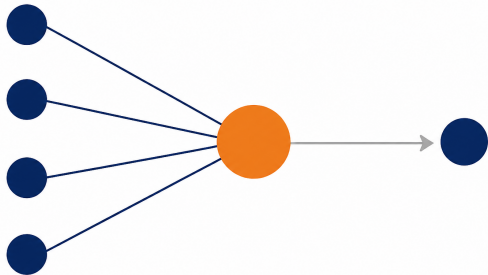
北京理工大学

学习目标

本讲要求：

- ① 用神经网络语言表示线性模型；
- ② 说明线性层、输出变换与损失的接口；
- ③ 理解梯度下降与 PyTorch 训练循环；
- ④ 解释无隐藏层网络的表示局限。

网络图



左侧圆圈 输入层，每个圆圈表示一个输入特征 $x^{(j)}$ ；

中间圆圈 神经元，汇总输入并完成一次计算；

连接线 信息传递方向，每条线对应一个权重 w_j ；

右侧圆圈 输出层，表示模型给出的预测结果。

箭头表示计算由输入端流向输出端。

神经元

定义 1 (神经元)

神经元是神经网络的基本计算单元：接收输入，计算加权和，再经输出变换产生结果。

对输入向量

$$x = (x^{(1)}, \dots, x^{(p)})^\top, \quad z = \sum_{j=1}^p w_j x^{(j)} + b = w^\top x + b, \quad h = \psi(z).$$

权重 w_j 控制第 j 个输入的作用；

偏置 b 调整整体输出水平；

变换 ψ 决定输出形式与含义。

网络层

层	作用	本讲中的形式
输入层	接收样本特征	$x = (x^{(1)}, \dots, x^{(p)})^\top$
隐藏层	学习中间表示	无
输出层	产生模型预测	$\hat{y} = \psi(w^\top x + b)$

无隐藏层网络中，输入直接连接到输出神经元：

$$x \longrightarrow z = w^\top x + b \longrightarrow \hat{y} = \psi(z).$$

说明

输入层只表示输入变量，通常不计为可训练层。本讲把这种结构称为“无隐藏层网络”。

矩阵形式

对 n 个样本组成的设计矩阵

$$X \in \mathbb{R}^{n \times p},$$

批量线性打分为

$$z = Xw + b\mathbf{1}_n \in \mathbb{R}^n.$$

多输出情形写成

$$Z = XW + \mathbf{1}_n b^\top, \quad W \in \mathbb{R}^{p \times q}, \quad b \in \mathbb{R}^q.$$

第一维是样本轴，第二维是输出轴。

旧模型的新表示

基于线性打分：

$$z = w^\top x + b.$$

任务	输出变换	损失	模型直接输出
实值回归	恒等映射	平方损失	$\hat{y} = z$
二分类	Sigmoid	交叉熵	logit z

本讲视角

第 2 讲已由概率模型推导损失；这里关注同一对应关系怎样嵌入神经网络计算图。

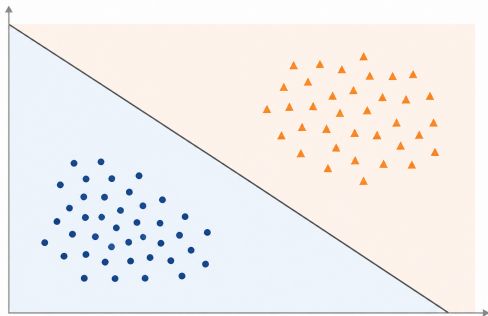
决策边界

取分类阈值 0.5:

$$\hat{y} = \begin{cases} 1, & p(x) \geq 0.5, \\ 0, & p(x) < 0.5. \end{cases}$$

因为 $\sigma(z) \geq 0.5$ 当且仅当 $z \geq 0$,
边界为

$$w^\top x + b = 0.$$



说明

概率 $p(x)$ 是输入的非线性函数，但决策边界仍是线性的超平面。

从经验风险到参数更新

第 2 讲已经把训练写成

$$\min_{\theta} R_{\text{emp}}(\theta).$$

一阶近似给出

$$R(\theta + \Delta\theta) \approx R(\theta) + \nabla R(\theta)^\top \Delta\theta.$$

因此负梯度是局部下降方向。

新增问题

损失已经定义后，怎样高效计算梯度并把它变成参数更新？

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla R_{\text{emp}}(\theta^{(t)})$$

η 学习率，控制更新步长；

t 迭代次数；

∇R 当前参数处的梯度。

学习率

学习率 η 控制每次参数更新的步长。

学习率	损失变化	典型结果
过小	缓慢下降	需要较多次迭代
合适	稳定下降	较快到达低损失区域
过大	反复振荡	可能无法收敛

注意

学习率是训练前设定的超参数，不是模型根据数据直接估计的参数。

线性层梯度

对批量线性打分

$$z = Xw + b\mathbf{1}, \quad \delta = \frac{\partial R}{\partial z},$$

参数梯度统一写成

$$\nabla_w R = \frac{1}{n} X^\top \delta, \quad \frac{\partial R}{\partial b} = \frac{1}{n} \mathbf{1}^\top \delta.$$

损失接口	输出误差信号 δ
平方损失	$\hat{y} - y$
BCEWithLogitsLoss	$\sigma(z) - y$

核心结构都是“输入表示乘以输出误差”。

算法步骤

给定训练集、初值 $\theta^{(0)}$ 、学习率 η 和最大迭代次数 T :

- ① 使用全部训练样本计算预测;
- ② 计算经验风险 $R_{\text{emp}}(\theta^{(t)})$;
- ③ 计算完整梯度 $\nabla R_{\text{emp}}(\theta^{(t)})$;
- ④ 更新参数

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla R_{\text{emp}}(\theta^{(t)});$$

- ⑤ 达到停止条件时结束，否则继续迭代。

常用停止条件包括：达到最大迭代次数、梯度足够小或风险变化足够小。

PyTorch 对应

两类模型都可用一个线性层表示：

```
model = nn.Linear(p, 1).
```

任务	损失函数	模型直接输出
线性回归	MSELoss	实值预测
逻辑回归	BCEWithLogitsLoss	logit

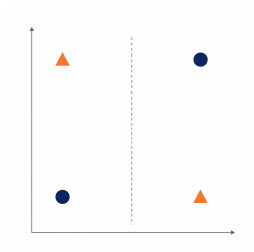
表示局限

无隐藏层网络只能利用

$$z = w^{\top} x + b$$

构造输出。

- 线性回归只能表达特征的线性组合；
- 逻辑回归只能形成线性决策边界；
- 两类样本交错分布，任一直线两侧都会混合类别。



虚线所示的单一直线不能分开两类。

下一步

引入隐藏层和非线性激活函数，学习输入的非线性表示。

理论小结

- ① 无隐藏层网络先计算线性打分 $w^\top x + b$;
- ② 输出变换与损失沿用第 2 讲的统计对应关系;
- ③ 线性层梯度由输入表示与输出误差信号共同决定;
- ④ 梯度下降沿负梯度方向迭代更新参数;
- ⑤ 没有隐藏层时, 模型仍受线性表示限制。

统一框架

线性打分 + 输出变换 + 损失函数 + 梯度下降.

实践任务

问题

同一个二分类模型，用不同训练与实现方式会得到什么结果？

实现	模型形式	梯度计算	参数更新
逻辑回归	$w^\top x + b$	解析公式	牛顿法
手写线性网络	$w^\top x + b$	解析公式	梯度下降
PyTorch 线性层	<code>nn.Linear</code>	自动微分	梯度下降

三种实现使用相同数据、交叉熵目标与 L_2 强度。

实验协议

- ① 固定随机种子，生成 $n = 500$ 、 $p = 12$ 的二分类数据；
- ② 划分训练集 375 个样本、测试集 125 个样本；
- ③ 用训练集估计标准化参数；
- ④ 固定 $\lambda = 0.001$ ，分别训练三种实现；
- ⑤ 比较参数、预测概率与测试指标。

控制变量

第 3 讲已经讨论超参数选择；这里固定 λ ，只比较实现与优化方式。

生成特征

用 4 个潜在变量构造 12 维相关特征：

```
# 固定种子
rng = np.random.default_rng(2026)
n, p = 500, 12

# 4 个潜在变量和 8 列噪声
latent = rng.normal(size=(n, 4))
noise = rng.normal(size=(n, p - 4))

# 后 8 列含共享信号和噪声
X = np.column_stack([
    latent, 0.65 * latent[:, :1] + 0.75 * noise
])
print("latent:", latent.shape, "noise:", noise.shape)
print("X:", X.shape)
```

生成标签

前 4 个特征决定类别概率，其余真实系数设为 0：

```
w_true = np.r_[
    2.2, -1.8, 1.5, -1.2, np.zeros(p - 4)
]
# 生成概率与 0/1 标签
logit = X @ w_true + 0.15
prob = sigmoid(logit)
y = rng.binomial(1, prob).astype(float)
print("y:", y.shape, "positive rate:", y.mean())
```

```
y: (500,) positive rate: 0.522
```

数据划分

固定样本顺序，将 25% 数据保留为测试集：

```
# 打乱后保留 25% 测试集
order = np.random.default_rng(2027).permutation(len(y))
test_idx, train_idx = order[:125], order[125:]

X_train, y_train = X[train_idx], y[train_idx]
X_test, y_test = X[test_idx], y[test_idx]

print("train:", X_train.shape, "test:", X_test.shape)
print(f"positive: {y_train.mean():.3f} {y_test.mean():.3f}")
```

```
train: (375, 12) test: (125, 12)
positive: 0.531 0.496
```

标准化

均值和标准差只能由训练数据估计：

```
# 按列估计训练集均值与标准差
mean = X_train.mean(axis=0)
scale = X_train.std(axis=0)

# 测试集复用训练集统计量
X_fit = (X_train - mean) / scale
X_eval = (X_test - mean) / scale

print("train mean:", np.round(X_fit.mean(0)[:3], 3))
print("train std:", np.round(X_fit.std(0)[:3], 3))
print("test mean:", np.round(X_eval.mean(0)[:3], 3))
```

同一个训练目标

三种实现都使用

$$p_i = \sigma(x_i^\top w + b), \quad \min_{w,b} \frac{1}{n} \sum_{i=1}^n [-y_i \log p_i - (1 - y_i) \log(1 - p_i)] + \frac{\lambda}{2} \|w\|_2^2.$$

比较重点

函数形式与目标完全相同；只改变梯度怎样获得、参数怎样更新。

参考解：牛顿法

```
beta = np.zeros(Xa.shape[1]) # 零初始化

for iteration in range(max_iter):
    p = sigmoid(Xa @ beta)
    grad = Xa.T @ (p - y) / n + lam * P @ beta

    # p(1-p) 给出每个样本的局部曲率
    r = np.clip(p * (1 - p), 1e-6, None)
    H = Xa.T @ (Xa * r[:, None]) / n + lam * P

    # 牛顿更新
    step = np.linalg.solve(H + 1e-8 * np.eye(len(beta)), grad)
    beta -= step
    if np.linalg.norm(step) < 1e-9:
        break
```

线性网络

将同一模型写成“线性层 + 概率输出”：

```
class LinearBinaryNet:
    def __init__(self, p):
        self.w = np.zeros(p)
        self.b = 0.0

    def forward(self, X):
        return X @ self.w + self.b

    def predict_proba(self, X):
        return sigmoid(self.forward(X))
```

forward 返回 $z = Xw + b$ ；predict_proba 返回 $p = \sigma(z)$ 。

梯度下降训练

对交叉熵，logit 的预测偏差简化为 $p - y$ ：

```
for epoch in range(2500):  
    # 前向传播  
    p = sigmoid(net.forward(X_fit))  
    error = p - y_train  
  
    # 交叉熵与 L2 梯度  
    grad_w = X_fit.T @ error / len(y_train) + lam * net.w  
    grad_b = error.mean()  
  
    # 参数更新  
    net.w -= 0.4 * grad_w  
    net.b -= 0.4 * grad_b
```

每次迭代都使用全部训练样本计算经验风险与梯度。

训练监控

在指定迭代位置记录目标函数和梯度长度：

```
if epoch in {0, 9, 99, 2499}:
    p_now = net.predict_proba(X_fit)
    objective = binary_log_loss(y_train, p_now)
    objective += 0.5 * lam * np.dot(net.w, net.w)
    grad_now = X_fit.T @ (p_now - y_train) / len(y_train)
    grad_now += lam * net.w
    print(epoch + 1, f"{objective:.4f}",
          f"{np.linalg.norm(grad_now):.4f}")
```

objective 含交叉熵与 L_2 惩罚；梯度长度衡量参数是否接近稳定。

训练输出

epoch	objective	gradient norm
1	0.5904	0.3596
10	0.4211	0.1391
100	0.3090	0.0217
2500	0.2912	0.0000

解释

目标函数持续下降，梯度逐渐接近 0；后期继续迭代带来的变化已经很小。

测试结果

实现	准确率	对数损失
牛顿法	0.832	0.428
手写梯度下降	0.832	0.428

观察

相同模型与目标充分收敛后，不同更新方法给出相同的测试表现。

参数对照

前四个标准化特征的估计权重为：

实现	w_1	w_2	w_3	w_4
牛顿法	2.804	-1.975	1.685	-1.377
手写梯度下降	2.803	-1.975	1.685	-1.377

$$\cos(w_{\text{logit}}, w_{\text{net}}) = 1.000000, \quad \max_i |p_i^{\text{logit}} - p_i^{\text{net}}| = 9.5 \times 10^{-5}.$$

优化路径不同，但参数方向和测试概率几乎完全一致。

PyTorch 前向

将同一目标交给 `nn.Linear`:

```
X_t = torch.as_tensor(X_fit, dtype=torch.float32)
y_t = torch.as_tensor(y_train, dtype=torch.float32)
model = nn.Linear(X_t.shape[1], 1)
loss_fn = nn.BCEWithLogitsLoss()
eta, lam = 0.1, 0.001

for epoch in range(2500):
    logit = model(X_t).squeeze(1)
    data_loss = loss_fn(logit, y_t)
    penalty = 0.5 * lam * model.weight.square().sum()
    loss = data_loss + penalty
```

`loss` 是全部训练样本上的标量目标函数。

PyTorch 反向

`backward()` 根据计算图自动得到各参数的梯度：

```
loss.backward()

print("loss shape:", loss.shape)
print("weight grad:", model.weight.grad.shape)
print("bias grad:", model.bias.grad.shape)
```

```
loss shape: torch.Size([])
weight grad: torch.Size([1, 12])
bias grad: torch.Size([1])
```

梯度形状与对应参数形状一致；此时参数值尚未改变。

PyTorch 更新

按梯度下降公式更新参数，并清空本轮梯度：

```
with torch.no_grad():  
    for parameter in model.parameters():  
        parameter -= eta * parameter.grad  
        parameter.grad.zero_()
```

对应关系

$$\text{parameter} \longleftrightarrow \theta, \quad \text{parameter.grad} \longleftrightarrow \nabla R_{\text{emp}}(\theta).$$

`torch.no_grad()` 表示参数更新本身不参与梯度计算。

课堂练习

配套脚本可用于完成以下练习：

- ① 比较手写梯度与 `parameter.grad`；
- ② 改变学习率，观察收敛速度与振荡；
- ③ 改变初始参数，比较最终预测；
- ④ 用 `torch.optim.SGD` 替换手工更新；
- ⑤ 加入一个隐藏层，测试 XOR 数据。

说明

每次只改变一个因素，并固定其他随机过程，才能解释实验差异。

实践小结

- ① 逻辑回归可直接写成一个无隐藏层网络；
- ② 手写实现与 `nn.Linear` 使用同一线性打分；
- ③ 自动微分替代求导过程，不改变训练目标；
- ④ 更新规则影响优化路径，不改变模型函数类；
- ⑤ 非线性边界需要隐藏层与激活函数。

实验主线

数据 → 线性打分 → 损失 → 梯度 → 参数更新.

几度沉浮



历史主线

同一个想法，经历了“被低估—被高估—被质疑—重新被理解”。

贯穿问题

为什么一个 1940 年代的想法，要等待近 70 年才改变计算机科学？

大胆假设

1943 年的问题

如果神经元只有“激活”和“不激活”两种状态，思维能否写成逻辑运算？

$$z = 1 \left\{ \sum_{j=1}^p w_j x_j \geq \theta \right\} \implies \text{AND、OR、NOT 等逻辑规则.}$$

- McCulloch 提出神经问题，Pitts 提供逻辑工具；
- 两人将神经活动写成命题逻辑。

著名故事

最初的目标并非识别图片，而是追问：思维能否被计算？

资料：McCulloch & Pitts, *Bulletin of Mathematical Biophysics* (1943).

感知机登场

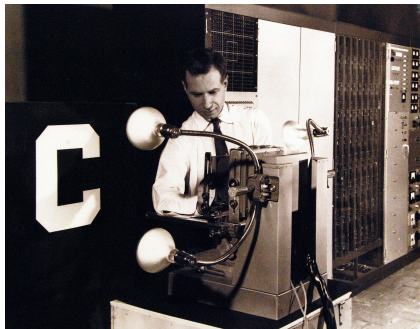
- 1957 年，Rosenblatt 提出感知机；
- 从 IBM 704 模拟走向 Mark I 硬件。

Mark I 使用光电感受器接收图案，并用电位器保存可调权重。

关键变化

规则不再全部由人编写；机器根据错误调整连接权重。

资料：Smithsonian National Museum of American History.



Mark I 感知机。图：U.S. Navy / Wikimedia Commons, Public Domain.

媒体狂热

1958 年海军新闻发布会之后，感知机迅速进入大众视野。

原型展示

- 从样本和反馈中调节权重；
- 区分简单图案或方向；
- 展示“机器可以学习”。

公众想象

- 机器将能够读图；
- 经验越多就越“聪明”；
- 很快接近人类感知能力。

著名故事

报纸把一次简单学习实验写成“会成长的电子大脑”。技术刚刚起步，预期已经冲到前面。

资料：The New York Times, “New Navy Device Learns by Doing” (1958); Cornell Chronicle.

XOR 反击

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

正类位于两个对角点：

$(0, 1), (1, 0).$

负类位于另外两个对角点：

$(0, 0), (1, 1).$

不存在一条直线将两类完全分开。

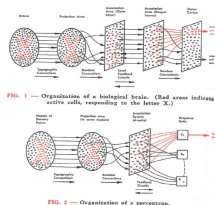
反转与出路

四行真值表揭示了单层感知机的边界。隐藏层可以组合多个线性边界，难点转为如何训练隐藏层。

一本书的阴影

1969 年，Minsky 与 Papert 的 *Perceptrons* 系统分析感知机能力。

- 单层模型的限制被严格说明；
- 多层网络可表达更多，却缺少有效训练方法；
- 热潮中的承诺遭遇现实边界。



Rosenblatt 的生物系统与感知机对照图，1958。Public Domain.

流行说法

“一本书杀死了神经网络。”

更准确的解释

理论边界、算力不足、数据有限、训练困难和过高预期共同造成低谷。

戛然而止

1971 年 7 月 11 日

Frank Rosenblatt 在切萨皮克湾的一次帆船事故中去世；当天正是他的 43 岁生日。

- 感知机刚遭遇强烈质疑，主要推动者又突然离场；
- 研究资金和学术注意力逐渐转向符号人工智能；
- 神经网络在 1970 年代退出舞台中央，但并未消失。

需要区分

这场悲剧并不是低谷的原因，却让早期神经网络的故事转折得格外突然。

资料：Emlen, Howland & O'Brien, *Frank Rosenblatt, 1928–1971*, Cornell University; Cornell Chronicle.

低谷火种

沉寂 多层网络仍在，但缺少有效训练方案；

1982 Hopfield 用能量函数构造联想记忆；

1986 Rumelhart、Hinton 与 Williams 展示误差反传与隐藏表示学习。

著名转折

反向传播所用的链式法则并不新；真正改变局面的，是它让多层网络能够学出有用特征。

历史含义

低谷不是技术停止，而是等待方法与条件重新汇合。

资料：Hopfield, *PNAS* (1982)；Rumelhart, Hinton & Williams, *Nature* (1986).

反向传播

- 前向计算得到预测和误差；
- 反向传播用链式法则逐层分配误差；
- 隐藏单元不再只是人工设计，而能从任务中形成表示；
- 1986 年的工作使这一训练方法广为人知。

重要区别

反向传播解决“怎样计算梯度”；梯度下降决定“怎样利用梯度更新参数”。

资料：Rumelhart, Hinton & Williams, *Nature* (1986).



Geoffrey Hinton 在 UBC 演讲，2013。图：Eviatar

Bach / Wikimedia Commons, CC BY-SA 3.0.

条件成熟

进入 2000 年代，三股力量逐渐汇合：

- ① **数据**：大规模标注数据集可以检验复杂模型；
- ② **算力**：GPU 擅长并行完成大量矩阵运算；
- ③ **算法**：激活函数、初始化、正则化与优化方法持续改进。



NVIDIA Tesla GPU。图：Mahogny / Wikimedia Commons, Public Domain.

变化

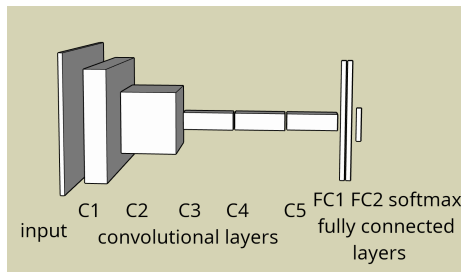
“能否训练”逐渐变成“如何更快、更稳地训练”。

资料：Krizhevsky, Sutskever & Hinton (2012).

2012 拐点

AlexNet 将深层卷积网络、大规模数据与 GPU 训练结合起来：

- 约 120 万张训练图像、1000 个类别；
- 5 个卷积层和 3 个全连接层；
- top-5 测试错误率为 15.3%，第二名为 26.2%。



AlexNet 结构示意图。图：Trebol6 / Wikimedia Commons, CC BY-SA 4.0.

影响

这不仅是一次比赛胜利，也展示了训练大型神经网络的可行工程路线。

数据：Krizhevsky, Sutskever & Hinton, NeurIPS (2012).

能力扩展

- 2016 AlphaGo 结合策略网络、价值网络与树搜索；
- 2017 Transformer 用注意力机制处理序列；
- 2020s 深度学习由识别扩展到语言、图像、音频与多模态生成。

趋势

模型从“识别已有内容”走向“理解、决策与生成”。

资料：Silver et al., *Nature* (2016)；Vaswani et al. (2017).



AlphaGo 展示机架，摄于 2025 年。图：Immigrant laborer /
Wikimedia Commons, CC0.

历史启示

早期思想 → 条件不足 → 方法复兴 → 规模化突破

- ① 好想法不一定立即成功，还需要数据、算力与工程条件；
- ② 低谷不等于研究路线被完全证伪；
- ③ 突破常来自旧思想与新条件的重新组合；
- ④ 面对热潮，既要保持兴趣，也要保留判断。

一句话总结

深度学习的历史，是理论、资源、工程与社会预期反复互动的历史。